We look at how to add authorization metadata to your endpoints in chapter 25, so for now we'll focus on improving the OpenAPI description of your app using metadata. You can provide a lot of details to document your APIs, some of which Swashbuckle uses during OpenAPI generation and some of which it doesn't. The following listing shows how to add a tag for each API and how to explicitly describe the responses that are returned, using `Produces()`.

Listing 11.2 Adding OpenAPI metadata to improve endpoint documentation

```
using System.Collections.Concurrent;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

WebApplication app = builder.Build();

var _fruit = new ConcurrentDictionary<string, Fruit>();

app.UseSwagger();
app.UseSwaggerUI();

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404))
    .WithTags("fruit")                                  ❶
    .Produces<Fruit>()                                  ❷
    .ProducesProblem(404);                              ❸

app.MapPost("/fruit/{id}", (string id, Fruit fruit) =>
    _fruit.TryAdd(id, fruit)
        ? TypedResults.Created($"/fruit/{id}", fruit)
        : Results.ValidationProblem(new Dictionary<string, string[]>
          {
              { "id", new[] { "A fruit with this id already exists" } }
        }))
    .WithTags("fruit")                                  ❹
    .Produces<Fruit>(201)                               ❺
    .ProducesValidationProblem();                       ❻

app.Run();
record Fruit(string Name, int stock);
```

❶ Adding a tag groups the endpoints in Swagger UI. Each endpoint can have multiple tags.

❷ The endpoint can return a Fruit object. When not specified, a 200 response is assumed.